

Лабораторная работа №11. Sealed-классы и моделирование состояний

Цель работы. Научиться:

- использовать **sealed-class** для описания состояний;
- безопасно обрабатывать варианты через **when**;
- применять sealed-классы в реальном проекте для моделирования событий и логики.

Продолжаем предыдущий проект.

Шаг 1. Sealed-классы базовая идея

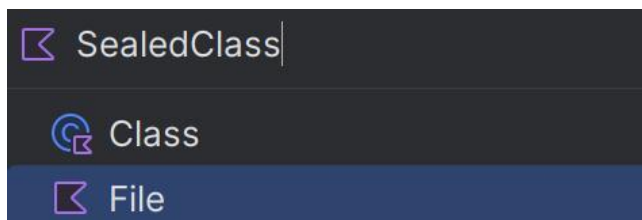
Sealed-классы (изолированные классы) — это специальный тип классов, которые **ограничивают наследование**. Они позволяют определить **закрытую иерархию** классов, где все возможные подклассы известны на этапе компиляции.

Основные характеристики:

1. **Ограниченное наследование** — можно наследовать только внутри того же файла или того же пакета;
2. **Все подклассы известны** — компилятор знает всех возможных наследников;
3. **Нельзя создать экземпляр** самого sealed-класса (он абстрактный);
4. **Идеально для представления ограниченного набора вариантов.**

Шаг 2. Пример sealed-класса: результат сетевого запроса

1. Создайте в папке **example** новый файл **SealedClass.kt**:



2. Определяем sealed-класс **NetworkResult**:

```
sealed class NetworkResult {  
}
```

Гарантирует, что **все варианты состояния будут обработаны**

Очень удобно для:

- o результатов сети
- o состояний UI
- o ошибок / успехов / загрузки

Важное правило: Все наследники sealed-class должны быть в том же файле.

3. Внутри класса определим наследников

3.1 Успешный результат

```
sealed class NetworkResult {  
    + data class Success(val data: String) : NetworkResult()  
}
```

data class — класс для хранения данных

data: String — результат успешного запроса

: NetworkResult() — говорит, что это **один из вариантов NetworkResult**

Пример: Запрос к серверу прошёл успешно, пришли данные

3.2 Ошибка

```
sealed class NetworkResult {  
    data class Success(val data: String) : NetworkResult()  
    + data class Error(val message: String, val code: Int) : NetworkResult()
```

message — текст ошибки

code — код ошибки (например, HTTP 404 / 500)

Тоже вариант **NetworkResult**

Пример: Сервер вернул ошибку 500

3.3 Загрузка

```
sealed class NetworkResult {  
    data class Success(val data: String)  
    data class Error(val message: String)  
    + object Loading : NetworkResult()
```

object — **одиночный экземпляр**. "**object** в **Kotlin** — это языковая конструкция, которая реализует **Singleton**. В **sealed**-классах его часто используют для состояний без данных, таких как **Loading** или **Empty**."

Нам не нужны данные — важен сам факт загрузки.

Пример: Запрос отправлен, ждём ответ

4. Для реализации нашего класса, создадим в текущем файле (за пределом класса) функцию **handleResult()**:

```
fun handleResult(result: NetworkResult) {  
}
```

В ней мы хотим принимать **любой** вариант **NetworkResult**.

Внутри функции используем конструкцию **when** без **else**:

```
fun handleResult(result: NetworkResult) {  
    when (result) {  
        // else НЕ НУЖЕН - компилятор знает все варианты!  
    }  
}
```

Здесь происходит **ключевая магия sealed-class**. Компилятор **точно знает**, какие варианты возможны.

Если мы наведём, то увидим о необходимости реализовать ветки с **Error**, **Loading**, **Success**:

```
! when (result) {  
'when' expression must be exhaustive. Add the 'is Error', 'Loading', 'is Success' branches or an 'else' branch.
```

Добавим их **внутри** нашей конструкции **when**:

4.1 Обработка Success

```
when (result) {  
+ {  
    is NetworkResult.Success -> {  
        println("Успех: ${result.data}")  
    }  
}
```

is — проверка типа

Kotlin автоматически **приводит тип**

Мы можем напрямую обратиться к `result.data`

Вывод: Успех: Данные получены

4.2 Обработка Error

```
is NetworkResult.Error -> {  
    println("Ошибка ${result.code}: ${result.message}")  
}
```

Получаем **code** и **message**

Удобно обрабатывать ошибки централизованно

Вывод: Ошибка 500: Сервер не отвечает

4.3 Обработка Loading

```
NetworkResult.Loading -> {  
    println("Загрузка...")  
}
```

Для **object** не нужен **is**

Это конкретный экземпляр

Вывод: Загрузка...

Почему не нужен else. Если вы **забудете обработать** один из вариантов — **Kotlin** выдаст **ошибку компиляции**, а не **runtime crash**. Это **огромный плюс sealed-class**.

5. Создадим внутри файла, точку входа main():

```
fun main() {  
  
}
```

Создадим объекты:

```
fun main() {  
    val success = NetworkResult.Success(data = "Данные получены")  
    val error = NetworkResult.Error(message = "Сервер не отвечает", code = 500)  
    val loading = NetworkResult.Loading
```

Мы создаём **три разных состояния**:

- успешный результат
- ошибка
- загрузка

Там же, внутри main() осуществим передачу в handleResult:

```
handleResult(result = success)  
handleResult(result = error)  
handleResult(result = loading)
```

Каждый объект: передаётся в **handleResult**; попадает в **when**; обрабатывается **своей веткой**.

Запускаем программу, и получаем следующий вывод:

```
Успех: Данные получены  
Ошибка 500: Сервер не отвечает  
Загрузка...
```

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "learn sealed-class"

git push

Шаг 3. Object в Kotlin

Object в Kotlin — это объявление синглтона, который существует только в одном экземпляре. Это как книга, у которой есть только одна копия в мире. Все, кто хочет прочитать эту книгу, читают одну и ту же.

Простая аналогия. **Представьте:**

- **Обычный класс** — это чертеж дома. По одному чертежу можно построить много домов.
- **Object** — это конкретный уникальный дом, который уже построен (например, Московский Кремль). Он один, и больше таких нет.

Ключевые особенности object

Создайте в папке **example** новый файл **ObjectClass.kt**.

Особенности:

1. Создается сразу при первом обращении

Рассмотрим на следующем примере, создайте в файле **object GameSession**:

```
object GameSession {  
    init {  
        println("Игровая сессия создана")  
    }  
    var isActive: Boolean = false  
    fun start() {  
        isActive = true  
        println("Игра началась")  
    }  
    fun end() {  
        isActive = false  
        println("Игра завершена")  
    }  
}
```

Проверяем в **main()** момент создания:

```
fun main() {  
    println("Программа запущена")  
    println("Проверяем состояние, но не трогаем GameSession")  
    println("Теперь запускаем игру")  
    GameSession.start()  
    println("Проверяем состояние ещё раз")  
    println("Активна ли сессия: ${GameSession.isActive}")  
}
```


Выводим в консоль:

```
Программа запущена
Проверяем состояние, но не трогаем GameSession
Теперь запускаем игру
Игровая сессия создана
Игра началась
Проверяем состояние ещё раз
Активна ли сессия: true
```

Обратите внимание:

- Пока мы не обращались к GameSession, объект не существовал
- Он был создан ровно в момент вызова GameSession.start()

2. Всегда один и тот же экземпляр

```
object Logger {
    var count = 0

    fun log(message: String) {
        count++
        println("[${count}] $message")
    }
}
```

И проверим в **main()**:

```
fun main() {
    Logger.log("Первое сообщение") // [1] Первое сообщение
    Logger.log("Второе сообщение") // [2] Второе сообщение
    // Это тот же самый объект!
    val logger1 = Logger
    val logger2 = Logger
    println(logger1 == logger2) // true - это один и тот же объект
}
```

3. Нельзя создать через конструктор

Где используется **object**?

1. Как синглтон (одиночка)

```
object AppSettings {
    val version = "1.0.0"
    var isDarkMode = true

    fun toggleTheme() {
        isDarkMode = !isDarkMode
    }
}
```

Все части программы используют один объект:

```
fun checkTheme() {  
    if (AppSettings.isDarkMode) {  
        println("Темная тема включена")  
    }  
}
```

2. В sealed-классах (как в нашем примере)

Зачем *object* в *sealed*-классе? Потому что *Loading* — это просто состояние, без дополнительных данных. Нет смысла создавать много одинаковых объектов "загрузка".

3. Для хранения констант или утилит

```
object Colors {  
    const val RED = "#FF0000"  
    const val GREEN = "#00FF00"  
    const val BLUE = "#0000FF"  
}  
  
fun main() {  
    println(Colors.RED)  
    println(Colors.GREEN)  
    println(Colors.BLUE)  
}
```

4. Object expression (анонимный объект)

Быстрое создание объекта "на лету"

```
fun main() {  
    val handler = object {  
        val name = "Обработчик"  
        fun handle() {  
            println("Обрабатываю...")  
        }  
    }  
    println(handler.name)  
    handler.handle()  
    // ...  
}
```

5. Разница с обычным классом

```
// Обычный класс - можно создавать много экземпляров
class MyCar(val model: String) {
    fun drive() = println("$model едет")
}
```

```
// Object - только один экземпляр
object TrafficController {
    var carCount = 0
    fun carPassed() {
        carCount++
    }
}
```

И в методе **main()**:

```
fun main() {
    MyCar(model = "Toyota")
    MyCar(model = "BMW") // Другой объект!
    TrafficController.carPassed() // Все обращаются к одному объекту
    //...
    //...
}
```

Итог:

- object создается лениво — только когда к нему обращаются в первый раз
- Потокобезопасность — создание гарантированно происходит один раз
- Используйте, когда нужен один экземпляр на всю программу

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "learn Object"

git push

Шаг 4. Состояние заказа

Внутри файла **SealedClass.k** создадим **sealed**-класс модели состояний:


```
sealed class OrderStatus {
    object Created : OrderStatus()
    object Paid : OrderStatus()
    object Shipped : OrderStatus()
    data class Cancelled(val reason: String) : OrderStatus()
}
```

Заказ в каждый момент времени находится ровно в одном статусе:

- **Created** — заказ создан
- **Paid** — оплачен
- **Shipped** — отправлен
- **Cancelled** — отменён (и у отмены есть причина)

Мы используем **object**, потому что:

- статус фиксирован
- нет данных

А для **Cancelled** используем **data class**, т.к. у отмены **есть данные** (причина)

Далее в нашем файле создадим функцию **handleOrder()**. Она будет выполнять логику обработки статуса:

```
fun handleOrder(status: OrderStatus) {
    when (status) {
        OrderStatus.Created -> println("Заказ создан")
        OrderStatus.Paid -> println("Заказ оплачен")
        OrderStatus.Shipped -> println("Заказ отправлен")
        is OrderStatus.Cancelled -> println("Отменён: ${status.reason}")
    }
}
```

И в методе **main()** реализуем (предыдущий код можете закомментировать):

```
fun main() {
    handleOrder(status = OrderStatus.Created)
    handleOrder(status = OrderStatus.Paid)
    handleOrder(status = OrderStatus.Shipped)
    handleOrder(status = OrderStatus.Cancelled(reason = "Нет товара на складе"))
    ...//прошлая реализация кода NetworkResult
}
```

Запускаем программу и получаем вывод:

```
Заказ создан
Заказ оплачен
Заказ отправлен
Отменён: Нет товара на складе
```

Главное правило:

- Если объект **не имеет данных и уникален по смыслу** → object
- Если объект **несёт данные** → data class
- Если вариантов немного, и они известны заранее → sealed-class

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "learn sealed-class"

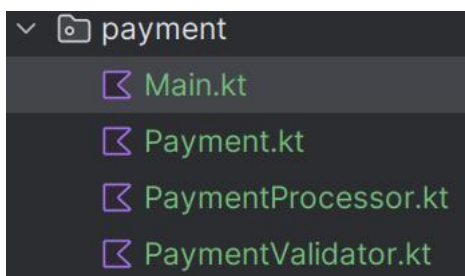
git push

Шаг 5. Система обработки платежей

Создайте новый пакет - *payment*



В нём создайте следующие файлы:



Следующий код пишем в файле **Payment.kt**

1. Итак, у нас есть задача - у вас есть система платежей.

Платёж может завершиться по-разному:

- успешно
- с ошибкой
- или быть в процессе обработки

Нам нужно, чтобы компилятор знал все возможные варианты результата и обработал их.

Здесь идеально подходит **sealed-class** - **PaymentResult**:

```
sealed class PaymentResult {}
```

Почему sealed?

- Все возможные подклассы **известны заранее**

- **when** по **PaymentResult** будет исчерпывающим
- Нельзя создать новый результат где-то «снаружи»

2. Теперь определяем варианты результата

2.1. Успех

Успешный платёж содержит идентификатор

```
sealed class PaymentResult {  
    +data class Success(val id: String) : PaymentResult()}
```

data class, потому что:

- есть данные
- нужен *toString*, *equals*, *copy*

2.2. Ошибка

Ошибка содержит причину

```
sealed class PaymentResult {  
    data class Success(val id: String) : PaymentResult()  
    +data class Error(val reason: String) : PaymentResult()  
}
```

2.3. В обработке

Это просто состояние, **без данных**, и оно должно быть **одно на всю программу**

```
sealed class PaymentResult {  
    data class Success(val id: String) : PaymentResult()  
    data class Error(val reason: String) : PaymentResult()  
    +object Processing : PaymentResult()  
}
```

Почему **object**, а не **class**?

- Нет полей
- Экземпляр один
- Не нужно создавать новые объекты

3. Теперь думаем о типах карт

Тип карты — это фиксированный набор значений

Других вариантов быть не может

Добавляем enum class:

```
enum class CardType {  
    VISA, MASTERCARD, MIR, UNKNOWN  
}
```

Почему enum?

- Ограниченный набор значений
- Удобно использовать в when
- Есть values(), ordinal, name

4. Описываем сам платёж

Платёж — это просто данные:

номер карты, сумма и тип карты

Добавляем data class:

```
data class Payment(  
    val card: String,  
    val sum: Int,  
    val type: CardType = CardType.UNKNOWN  
)
```

Почему data class?

Мы хотим:

- сравнивать платежи
- копировать (copy)
- красиво печатать

*Закончили с файлом **Payment.kt**. Теперь переходим к написанию файла **PaymentValidator.kt***

5. Валидатор платежа

Перед оплатой я хочу проверить корректность данных

```
class PaymentValidator {  
    fun check(payment: Payment): Boolean {  
        return when (payment.type) {  
            CardType.VISA -> payment.card.length == 16  
                && payment.card.startsWith(prefix = "4")  
            CardType.MASTERCARD -> payment.card.length == 16  
                && payment.card.startsWith(prefix = "5")  
            CardType.MIR -> payment.card.length == 16  
                && payment.card.startsWith(prefix = "2")  
            CardType.UNKNOWN -> payment.card.length == 16  
        } && payment.sum > 0  
    }  
}
```

Закончили с файлом **PaymentValidator.kt**. Теперь переходим к написанию файла **PaymentProcessor.kt**

6. Процессор платежей

Процессор:

- валидирует платёж
- обрабатывает его
- возвращает результат (PaymentResult)

```
class PaymentProcessor {  
    private val validator = PaymentValidator()  
}
```

Далее: Если платёж неправильный — ошибка. Иначе результат зависит от типа карты.

Пишем метод **pay()** внутри класса:

```
fun pay(payment: Payment): PaymentResult {  
    if (!validator.check(payment)) {  
        return PaymentResult.Error(reason = "Ошибка валидации")  
    }  
  
    return when (payment.type) {  
        CardType.VISA -> PaymentResult.Success(id = "VISA-${System.currentTimeMillis()}")  
        CardType.MASTERCARD -> PaymentResult.Processing  
        CardType.MIR -> PaymentResult.Success(id = "MIR-${System.currentTimeMillis()}")  
        CardType.UNKNOWN -> PaymentResult.Error(reason = "Неизвестный тип карты")  
    }  
}
```

Обратите внимание:

- Метод возвращает sealed-class
- when гарантирует, что ты обработал все варианты CardType

Думаем дальше:

Нужно красиво вывести результат, не зная заранее, какой он

Пишем метод **show()** внутри класса:

```
fun show(result: PaymentResult) {  
    when (result) {  
        is PaymentResult.Success -> println("Успех: ${result.id}")  
        is PaymentResult.Error -> println("Ошибка: ${result.reason}")  
        PaymentResult.Processing -> println("В обработке...")  
    }  
}
```


Главное преимущество sealed-class:

- when без else
- Компилятор гарантирует безопасность

7. main — связываем всё вместе

Переходим в **Main.kt**. Далее:

- Создаём процессор
- Создаём список платежей
- Обрабатываем каждый

```
fun main() {  
    val processor = PaymentProcessor()  
    val payments = listOf(  
        Payment(card = "4_111_111_111_111_111", sum = 1000, type = CardType.VISA),  
        Payment(card = "5_111_111_111_111_111", sum = 2000, type = CardType.MASTERCARD),  
        Payment(card = "2_222_222_222_222_222", sum = 1500, type = CardType.MIR),  
        Payment(card = "1234567812345678", sum = 500, type = CardType.UNKNOWN),  
        Payment(card = "123", sum = 3000, type = CardType.VISA) // Неправильная длина  
    )  
    println("=== Обработка платежей ===")  
    payments.forEach { payment ->  
        println("\nПлатеж ${payment.type}: ${payment.card.take(n = 4)}..., ${payment.sum} py6")  
        val result = processor.pay(payment)  
        processor.show(result)  
    }  
}
```

8. Демонстрация enum

Допишем внутри **main()**:

```
println("\n=== Работа с enum ===")  
val cardType = CardType.VISA  
println("Тип карты: $cardType")  
println("Порядковый номер: ${cardType.ordinal}")  
println("Все типы карт: ${CardType.values().joinToString()}")
```

9. Демонстрация data class

Допишем внутри **main()**:

```
val payment1 = Payment(card = "4111111111111111", sum = 1000, type = CardType.VISA)  
val payment2 = payment1.copy(type = CardType.MASTERCARD, sum = 2000)  
  
println("\n=== Сравнение data class ===")  
println("Платеж 1: $payment1")  
println("Платеж 2: $payment2")  
println("Одинаковые? ${payment1 == payment2}")
```


Итоговая картина:

- **sealed-class** - “Закрытая иерархия состояний”
- **object** - “Одиночное состояние без данных”
- **enum** - “Ограниченный список вариантов”
- **data class** - “Просто данные + удобство”

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

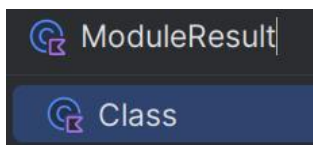
git add .

git commit -m "Create payment processing system"

git push

Шаг 6. Применяем Sealed-классы в Galaxy Outpost Manager

Создадим класс **ModuleResult** в пакете **modules** - результат работы модуля базы:



В нём мы создадим модель домена:

- **Success** - универсальный успешный результат;
- **ResourceProduced** - частый кейс для генераторов;
- **NotEnoughResources** - классическая ошибка менеджмента ресурсов;
- **Error** - защита на будущее.

```
sealed class ModuleResult {  
    data class Success(val message: String) : ModuleResult()  
    data class ResourceProduced(  
        val resourceName: String, val amount: Int  
    ) : ModuleResult()  
    data class NotEnoughResources(  
        val resourceName: String,  
        val required: Int,  
        val available: Int  
    ) : ModuleResult()  
    data class Error(val reason: String) : ModuleResult()  
}
```

Далее в абстрактном классе **OutpostModule** изменим:

```
abstract fun performAction(manager: ResourceManager) : ModuleResult
```

Теперь `performAction()` возвращает результат, а не просто печатает текст.

Внесём изменения в класс **EnergyGenerator**:

```
class EnergyGenerator : OutpostModule(name = "Генератор энергии") {  
    override fun performAction(manager: ResourceManager) : ModuleResult {  
        println("Генератор работает...производит 20 энергии")  
        val energy = manager.get("Energy")  
        return if (energy != null) {  
            energy.amount += 20  
            + ModuleResult.ResourceProduced(resourceName = "Energy", amount = 20)  
        } else {  
            manager.add(OutpostResource(id = 99, name = "Energy", amount = 20))  
            + ModuleResult.Success(message = "Энергия создана впервые")  
        }  
    }  
}
```

И в класс **ResearchLab**:

```
class ResearchLab : OutpostModule(name = "Исследовательская лаборатория") {  
    override fun performAction(manager: ResourceManager) : ModuleResult {  
        val minerals = manager.get("Minerals")  
        if (minerals == null || minerals.amount < 30) {  
            return ModuleResult.NotEnoughResources(  
                resourceName = "Minerals",  
                + required = 30,  
                available = minerals?.amount ?: 0  
            )  
        }  
        minerals.amount -= 30  
        + return ModuleResult.Success(message = "Исследование завершено")  
    }  
}
```

Дальше добавим функцию обработки результатов **handleModuleResult()** в файл **Main.kt**:

```

fun handleModuleResult(result: ModuleResult) {
    when (result) {
        is ModuleResult.Success ->
            println("УСПЕХ: ${result.message}")
        is ModuleResult.ResourceProduced ->
            println("Произведено: ${result.resourceName} + ${result.amount}")
        is ModuleResult.NotEnoughResources ->
            println(
                "Недостаточно ресурса ${result.resourceName}. " +
                "Нужно: ${result.required}, есть: ${result.available}"
            )
        is ModuleResult.Error ->
            println("ОШИБКА: ${result.reason}")
    }
}

```

Осталось внести небольшие правки в метод **main()**. После объявления переменных **generator** и **lab**, сохраняем результаты выполнения модулей:

```

val generatorResult = generator.performAction(manager)
val labResult = lab.performAction(manager)

```

Далее обрабатываем результаты через **handleModuleResult**:

```

handleModuleResult(generatorResult)
handleModuleResult(labResult)

```

И как и раньше выводим результат:

```

println()
manager.printAll()

```

Смотрим вызов в терминал:

```

Добавлен ресурс: Minerals
Добавлен ресурс: Gas
Генератор работает...производит 20 энергии
Добавлен ресурс: Energy
УСПЕХ: Энергия создана впервые
УСПЕХ: Исследование завершено

Ресурсы базы
Minerals: 270
Gas: 100
Energy: 20

```

Теперь у нас:

- централизованно обрабатываются все типы результатов;
- добавляются новые варианты без изменений всей логики;
- легко интегрируется с UI, логами или системой событий.

Сохранение изменений в системе контроля версий

После завершения работы с выполните следующие команды в терминале:

git add .

git commit -m "Create sealed model ModuleResult"

git push

Шаг 7. Добавление описания в README.md

В файле README.md добавьте описание изученных тем и примеры их применения в Kotlin.

Темы:

- Sealed-классы
- Object в Kotlin

Примерный текст для вставки (можете добавить свои примеры):

Galaxy Outpost Manager

Учебный проект на Kotlin, демонстрирующий основы объектно-ориентированного программирования и архитектурные приёмы языка.

Sealed-классы

Sealed-классы используются для представления ограниченного набора состояний или результатов, которые известны на этапе компиляции.

Они позволяют:

гарантировать обработку всех возможных вариантов;

безопасно использовать конструкцию when без else;

удобно описывать состояния, события и результаты действий.

Пример: результат работы модуля

sealed-class ModuleResult {

data class Success(val message: String) : ModuleResult()

data class ResourceProduced(val resourceName: String, val amount: Int) : ModuleResult()

data class NotEnoughResources(

val resourceName: String,

val required: Int,

val available: Int

) : ModuleResult()

data class Error(val reason: String) : ModuleResult()

```
}
```

Object в Kotlin

object — это специальная конструкция Kotlin, которая создаёт единственный экземпляр класса (Singleton).

Особенности:

создаётся при первом обращении;

существует в одном экземпляре;

не имеет конструктора.

Пример: глобальный логгер

```
object Logger {
```

```
    private var counter = 0
```

```
    fun log(message: String) {
```

```
        counter++
```

```
        println("[${counter}] $message")
```

```
    }
```

```
}
```

Использование:

```
Logger.log("Инициализация системы")
```

```
Logger.log("Модуль запущен")
```

object удобно использовать для:

логгеров;

конфигураций;

состояний без данных в sealed-классах;

утилитарных классов.

Пример оформления:

README



Galaxy Outpost Manager

Учебный проект на Kotlin, демонстрирующий основы объектно-ориентированного программирования и архитектурные приёмы языка.

Sealed-классы

Sealed-классы используются для представления ограниченного набора состояний или результатов, которые известны на этапе компиляции.

Они позволяют:

- гарантировать обработку всех возможных вариантов;
- безопасно использовать конструкцию `when` без `else`;
- удобно описывать состояния, события и результаты действий.

Пример: результат работы модуля

```
sealed class ModuleResult {  
    data class Success(val message: String) : ModuleResult()  
    data class ResourceProduced(val resourceName: String, val amount: Int) : ModuleResult()  
    data class NotEnoughResources(  
        val resourceName: String,  
        val required: Int,  
        val available: Int  
    ) : ModuleResult()  
    data class Error(val reason: String) : ModuleResult()  
}
```



Object в Kotlin

object — это специальная конструкция Kotlin, которая создаёт единственный экземпляр класса (Singleton).

Особенности:

- создаётся при первом обращении;
- существует в одном экземпляре;
- не имеет конструктора.

Пример: глобальный логгер

```
object Logger {  
    private var counter = 0  
  
    fun log(message: String) {  
        counter++  
        println("[${counter}] $message")  
    }  
}
```



Использование:

```
Logger.log("Инициализация системы")  
Logger.log("Модуль запущен")
```



object удобно использовать для:

- логгеров;
- конфигураций;
- состояний без данных в sealed-классах;
- утилитарных классов.

После внесения изменений выполните команды:

git add .

git commit -m "docs: add sealed-classes and object description"

git push

Самостоятельное задание

Тема: Игровая система состояний персонажа

Структура проекта

Все файлы должны находиться в одном пакете:

gameCharacter

Шаг 1. Создание sealed-класса состояний персонажа

1. В пакете **gameCharacter** создайте файл **CharacterState.kt**.
2. Объявите **sealed-class**, который будет описывать состояние персонажа в игре.
3. Реализуйте следующие состояния:

➤ **Бездействие**

- не содержит данных;
- реализуется через object.

➤ **Бег**

- не содержит данных;
- реализуется через object.

➤ **Атака**

- содержит числовое значение урона;
- реализуется через data class.

➤ **Смерть**

- содержит строковую причину смерти;
- реализуется через data class.

Подсказка:

Используйте **object**, если состояние не имеет данных,
и **data class**, если состояние хранит информацию.

Шаг 2. Класс игрового персонажа

В пакете **gameCharacter** создайте файл **GameCharacter.kt**.

Создайте класс персонажа, который:

- хранит имя персонажа;
- имеет текущее состояние (**CharacterState**);
- по умолчанию находится в состоянии **бездействия**.

Добавьте метод для изменения состояния персонажа.

Шаг 3. Обработка состояния персонажа

1. В том же пакете создайте файл **Main.kt**.

2. В **main()**:

- создайте объект персонажа;
- последовательно переведите его в разные состояния:
- бег;
- атака (с указанием урона);
- смерть (с причиной).

3. Для обработки состояния создайте отдельную функцию, которая:

- принимает **CharacterState**;
- использует **when** без **else**;
- выводит сообщение в консоль для каждого состояния.

Пример сообщений:

- «Персонаж бездействует»
- «Персонаж бежит»
- «Персонаж атакует с уроном X»
- «Персонаж погиб: причина»

Шаг 4. Контроль версий

После выполнения задания выполните команды:

git add .

git commit -m "add character state system using sealed-classes"

git push